Pascal Rows in Mersenne Bases and the Largest Carry-Free Row

Abstract

This explanation examines how Pascal rows can serve as digit sequences in a positional base, and where that representation stops being carry-free. The first question is whether a selected row fits the base. The second is which row is the last one that fits. The first study selects n = p + 2. The second keeps the base fixed and varies the row index. The last valid row index is denoted by N(p).

Both questions use the same binomial identity. They do not, however, ask for the same result. For p ≥ 7, row p + 2 is known to be valid. It need not be the last valid row. At p = 38, row p + 2 = 40 is valid, but row 41 is the last valid one.

The explanation starts with bases and digits, then introduces Pascal's triangle, carrying, worked examples, base selection and the thresholds of N(p). Proved results, finite computational checks and an open conjecture are kept distinct. Detailed proofs are in the original mathematical note and its N(p) extension [1, 2].

Keywords: Pascal's triangle, Mersenne base, binomial coefficient, carry-free representation, central coefficient.

Symbol	Meaning
p	Integer parameter defining the base
$b = 2^p - 1$	The positional base
n	Index of the Pascal row under consideration
C(n,k)	Binomial coefficient at position k in row n
$c_n = C(n,\lfloor n/2 \rfloor)$	Largest coefficient in row n
N(p)	Index of the last carry-free row in base b
D(p) = N(p) − p	Difference between the last row index and p

1. Bases and digits

1.1. Writing a number as digits

In base ten, 347 expands as 3 × 100 + 4 × 10 + 7. Both the digit values and their positions matter. The same rule holds in base b. The digits a_2, a_1, a_0, read from left to right, represent $a_2b^2 + a_1b + a_0$.

A valid digit is an integer from 0 to b − 1. Once a digit value reaches b, a carry moves to the next position. The fact that 10 is not a single decimal digit is one instance of this rule.

For large bases, we can write digit values in decimal and separate them with commas. In base 127, the value 126 is one digit. Having three symbols in its decimal spelling does not make it three base-127 digits.

Base and digit sequence	Decimal value
Base 127: [1, 2]	1 × 127 + 2 = 129
Base 127: [1, 126]	1 × 127 + 126 = 253
Base 127: [1, 127]	The second value is not a valid digit. Carrying gives [2, 0].

1.2. What carry-free means

A coefficient sequence is a carry-free representation when every coefficient is smaller than the base. Expansions with and without carrying can have the same numerical value. Once carrying occurs, however, the original coefficients no longer remain the actual digit sequence of that number.

The property studied here is not numerical equality alone. It is whether Pascal coefficients can be used unchanged as actual digits. The base constraint is therefore strict: $C(n,k) < b$ for every k.

2. Pascal rows and the largest coefficient

Pascal's triangle starts at row $n = 0$. Both ends of each row are 1. Each interior entry is the sum of the two neighboring entries above it. Row n contains $n + 1$ coefficients.

Row index	Coefficients
0	1
1	1, 1
2	1, 2, 1
3	1, 3, 3, 1
4	1, 4, 6, 4, 1
5	1, 5, 10, 10, 5, 1

The rows are symmetric. The largest coefficient is in the middle. An even row index gives a single center; an odd row index gives two equal central coefficients. For example, row 4 has center 6, and row 5 has two central values of 10.

This simplifies the carry-free check. If the largest coefficient is smaller than the base, all the others are too. It is enough to compare $c_n = C(n,\lfloor n/2 \rfloor)$ with the base instead of comparing every entry separately.

For $n \geq 1$, central coefficients strictly increase. Once the base limit is reached, later rows cannot become carry-free again in that same base. This monotonicity makes the last carry-free row well defined.

3. Pascal coefficients and powers

3.1. The binomial identity

The binomial theorem gives the coefficients of $(b + 1)^n$ from Pascal's triangle. For $n = 4$,

$(b + 1)^4 = b^4 + 4b^3 + 6b^2 + 4b + 1$.

Its coefficient sequence is [1, 4, 6, 4, 1]. When every coefficient is smaller than b, this sequence directly gives the base-b digits of the result. Otherwise the identity remains true, but carrying changes the digits.

In base ten, $11^2 = 121$, $11^3 = 1331$ and $11^4 = 14641$ illustrate the relationship. Yet $11^5 = 161051$. The two 10s in Pascal row 5 cannot be decimal digits, so [1, 5, 10, 10, 5, 1] is not preserved directly.

3.2. Choosing a Mersenne base

Choose $b = 2^p - 1$. Then $b + 1 = 2^p$, and the identity becomes $(b + 1)^n = 2^{pn}$. If the carry-free condition holds, the base-b digits of 2^{pn} are exactly Pascal row n.

Here $p \geq 2$ is an integer parameter. Neither p nor $2^p - 1$ needs to be prime for this general carry-free question. The later example p = 38 is composite. This representation is not a primality test.

Two statements must be kept separate. The numerical identity holds for every valid b and n. Keeping the coefficients unchanged as digits additionally requires $c_n < b$.

3.3. Seeing a carry in a small example

With p = 3, the base is 7. Pascal row 4, [1, 4, 6, 4, 1], fits this base. Its value can be calculated by hand:

$1 \times 7^4 + 4 \times 7^3 + 6 \times 7^2 + 4 \times 7 + 1 = 2401 + 1372 + 294 + 28 + 1 = 4096$.

This equals $8^4 = 2^{12}$. Row 5 is [1, 5, 10, 10, 5, 1]. In base 7, 10 cannot be a single digit. Carrying from right to left changes the sequence:

Operation	Digit left in place	Carry to the left
Right-hand 10 = 1 × 7 + 3	3	1
Left-hand 10 + 1 = 11 = 1 × 7 + 4	4	1
Neighboring 5 + 1 = 6	6	0

The result is [1, 6, 4, 3, 5, 1]. Its value is still $8^5 = 32768$. What is lost is not the numerical identity, but the original Pascal row as the actual digit sequence. Thus N(3) = 4. The definition applies to p = 3, while the p + 2 guarantee applies only to $p \geq 7$.

4. The row selected by the first study

The original Pascal Mersenne Rows study considers n = p + 2 [1]. It proves that this row is carry-free in base $2^p - 1$ for every integer $p \geq 7$. This gives a general validity guarantee for one selected row.

For p = 7, the base is 127 and n = 9. Row 9 is

```
1 9 36 84 126 126 84 36 9 1
```

Its largest coefficient is 126. Since 126 < 127, all ten coefficients are valid digits. Their sum with the positional weights is $2^{7\times9} = 2^{63}$.

Row	Largest coefficient	Status in base 127
8	70	Carry-free
9	126	Carry-free
10	252	Carrying required

In this example the selected row 9 is also the last carry-free row. Thus N(7) = 9. This one example does not show that p + 2 is always the last valid row.

The original note already contains this distinction. For $7 \leq p \leq 37$, p + 2 is the last carry-free row. It also observes that p + 3 is valid at p = 38. The N(p) study systematically extends this boundary question, rather than introducing a distinction entirely absent from the earlier note.

5. The N(p) question and p = 38

5.1. Keeping the base fixed while varying the row

N(p) is the largest integer n satisfying $C(n,\lfloor n/2 \rfloor) < 2^p - 1$. In computing N(p), p and its base remain fixed. Only the row index varies.

For p = 38, the base is 274877906943. The first study selects row 40. Its central coefficient and those of the next two rows are compared below.

Row n	Central coefficient c_n	Result
40	137846528820	Smaller than the base
41	269128937220	Smaller than the base
42	538257874440	Larger than the base

Therefore N(38) = 41. The first study guarantees row 40. The boundary study determines that one more row is possible in this same base. The two results are compatible.

5.2. The same base, different numerical values

The digits of row 40 represent $2^{38\times40} = 2^{1520}$. Those of row 41 represent $2^{38\times41} = 2^{1558}$. The base is the same, but the numbers represented are different.

N(p) is not the largest number writable in that base. Any base can represent larger numbers by using more digits. N(p) is only the boundary for the family of Pascal rows whose coefficients remain unchanged by carrying.

6. Thresholds and the difference between the studies

6.1. The stepwise change in the last row

D(p) = N(p) − p stays constant over certain ranges. The first ranges are [2]:

Range of p	Last carry-free row	Extra rows beyond p + 2
7-37	N(p) = p + 2	0
38-158	N(p) = p + 3	1
159-645	N(p) = p + 4	2
646-2600	N(p) = p + 5	3
2601-10421	N(p) = p + 6	4
10422-41712	N(p) = p + 7	5

For example, N(100) = 103 and N(1000) = 1005. The initially selected rows are 102 and 1002. D(p) is not the same as the number of additional rows beyond the first study's selection. That number is N(p) − (p + 2), or D(p) − 2.

Moving from N(37) = 39 to N(38) = 41 does not mean row 40 is skipped within a fixed base. Changing p changes the base. These two N values refer to different bases.

6.2. The exact threshold definition

For $k \geq 2$, let m_k be the smallest integer m satisfying $2^k C(2m,m) < 4^m$. The corresponding threshold is $p_k = 2m_k - k - 1$. For $p \geq 7$, D(p) is the largest k for which $p_k \leq p$. The detailed derivation is in the N(p) note [2].

An exact definition is different from a proved short closed formula. The expression $m_k = \lceil 4^k/\pi - 1/4 \rceil$ is a computationally checked candidate. The checks for k = 1,…,12 support it. It is not treated as proved for every k in this work.

Topic	Original study	N(p) extension
Question	Is row p + 2 valid?	Which row is the last valid one?
Row selection	n = p + 2 is fixed in advance.	n varies until the boundary is found.
Shared basis	Binomial identity and central coefficient	The same identity and coefficient
Kind of result	General guarantee for a selected row	Last row, thresholds and growth bounds

7. Approximate size and the scope of the proof

The size of central binomial coefficients leads to [2]

$N(p) = p + (1/2)\log_2(\pi p/2) + E(p)$.

For $p \geq 7$, the error satisfies $-1 < E(p) \leq 2 - (1/2)\log_2(7\pi/2)$. The upper endpoint is approximately 0.270574474235. The decimal 0.270575 rounds it outward. The exact endpoint and the rounded decimal are not identical.

The relation shows that the increment above p grows slowly. At p = 1000, the expression without the error is approximately 1005.309. The exact answer is 1005. Rounding to a nearby integer in one example does not prove a universal rounding rule.

The package directly checks p = 7,…,20000 and thresholds k = 1,…,12. This is a finite check. The proved threshold characterization together with exact computed thresholds can imply results over a wider range. Such an implication does not mean every p in that wider range was individually scanned.

Some Catalan and factor relations in the original study depend on odd p and the choice n = p + 2. Replacing that row by N(p) does not automatically preserve these conditions. The original special relations should not be transferred to the new row without an additional proof.

8. Ten examples: what is N(p) useful for?

N(p) gives the validity boundary when Pascal coefficients are used as the digits of a number. With the base fixed, it tells us how far we can go through the rows. In the reverse direction, it helps select a sufficient base for a desired row. The examples below illustrate these uses. They are not new speed or compression results.

Example 1. A number small enough to check by hand

For p = 3 the base is 7. Pascal row 4 is [1, 4, 6, 4, 1]. Its largest coefficient is 6 < 7, so all entries are valid digits. Indeed,

$1 \times 7^4 + 4 \times 7^3 + 6 \times 7^2 + 4 \times 7 + 1 = 4096 = 8^4 = 2^{12}$.

Row 5 is [1, 5, 10, 10, 5, 1]. The value 10 is not a base-7 digit. The coefficient expansion still equals $8^5 = 32768$, but the actual digits are [1, 6, 4, 3, 5, 1]. The right-hand 10 gives remainder 3 and carry 1. The next 10

becomes 11 after that carry, giving remainder 4 and carry 1. Its neighboring 5 becomes 6. Thus N(3) = 4. This p = 3 example illustrates the definition; it is outside Theorem A, which requires p ≥ 7.

Example 2. Equality at the boundary also fails

For p = 2 the base is 3. Row 2, [1, 2, 1], fits. Row 3, [1, 3, 3, 1], does not, because the only valid digits are 0, 1 and 2. Hence N(2) = 2. The check must be c(n) < b, not c(n) ≤ b. A program accepting equality would produce an invalid digit sequence.

Example 3. Seeing the complete row in base 127

For p = 7 the base is 127. Row 9 is [1, 9, 36, 84, 126, 126, 84, 36, 9, 1] and represents 2^{63}. Its central coefficient is 126 < 127; the next row has central coefficient 252. Therefore N(7) = 9. In this example the first note's choice p + 2 reaches the boundary exactly.

Example 4. Going beyond the guaranteed row

For p = 38 the situation is different:

Row	Central coefficient	Comparison with base 274877906943
40	137846528820	Smaller, coefficients remain digits
41	269128937220	Smaller, coefficients remain digits
42	538257874440	Larger, carrying is required

N(38) = 41. Thus p + 2 = 40 is a safe choice, but not the last valid choice. A converter can accept row 41 in this base; it cannot accept the coefficients of row 42 directly as digits.

Example 5. Choosing a base for a desired row

Suppose we want to write row 42 without carrying. Its central coefficient is 538257874440. The p = 38 base is insufficient. For p = 39 the base is 549755813887, larger than the central coefficient. Therefore 39 is the smallest sufficient p among Mersenne bases. The next central coefficient, 1052049481860, also confirms N(39) = 42.

The general decision rule is to choose the smallest p ≥ 2 satisfying c(n) < 2^p − 1 for the desired row n. This reverses the question of finding N(p) for a fixed p. It is sufficient to check the largest coefficient rather than every entry separately.

Example 6. A number longer than one million bits

For p = 1000 the base is b = 2^{1000} − 1 and the last row is N(1000) = 1005. The beginning and end of that row are

[1, 1005, 504510, …, 504510, 1005, 1].

There are 1006 digits in total. The … represents 1000 omitted Pascal coefficients, not zeros. The largest coefficient has 301 decimal digits. All coefficients are below the base. With their positional weights, these digits represent $2^{1\,005\,000}$.

In binary, this number is a single 1 followed by 1 005 000 zeros. Its exact binary length is therefore 1 005 001 bits. The use here is to know precisely which Pascal row represents a large number without carrying. There is no need to print its entire decimal expansion.

Using 1006 digits does not by itself compress data. Each digit may require up to 1000 bits. The number of digits and the number of bits required to store them are different quantities.

Example 7. A number whose raw binary form needs about 125 GB

Choose p = 1 000 000. Since the thresholds give 667533 ≤ p < 2670166, we have N(p) − p = 10 and N(p) = 1 000 010. The base is $b = 2^{1\,000\,000} - 1$.

[1, 1000010, 500009500045, …, 500009500045, 1000010, 1].

The row contains 1 000 011 digits. The omitted middle contains 1 000 005 coefficients. The represented number is $2^{1\,000\,010\,000\,000}$, with exact binary length 1 000 010 000 001 bits.

Storing those bits without compression in eight-bit bytes requires 125 001 250 001 bytes, about 125 GB. By contrast, “2 raised to 1 000 010 000 000” is a short symbolic description. A symbolic description and expanding every bit in memory are different operations. This example does not generate the entire huge number or row. The last row is determined from the mathematical threshold result.

Example 8. A parameter of one trillion

Let $p = 10^{12} = 1\,000\,000\,000\,000$. At this scale we can use the proved error band instead of trying every row. For $h(p) = \tfrac{1}{2}\log_2(\pi p/2)$, we have $20.257 < h(p) < 20.258$. Combining this with $-1 < E(p) \le 0.270575$ from the mathematical note gives

$19.257 < N(p) - p < 20.529$.

N(p) − p is an integer. The only integer in this interval is 20. Therefore $N(p) = 10^{12} + 20 = 1\,000\,000\,000\,020$.

Writing $n = 10^{12} + 20$, the row has the form [1, n, n(n − 1)/2, …, n(n − 1)/2, n, 1]. There are $10^{12} + 21$ digits. The represented number is $2^{10^{24} + 20 \times 10^{12}}$. This does not mean every coefficient was computed individually. The last row was determined by selecting the unique integer permitted by the proved bound.

Example 9. When p itself has 101 decimal digits

Now let $p = 10^{100}$. In decimal, this is a single 1 followed by 100 zeros. The base is $b = 2^{10^{100}} - 1$. We have $166.422 < h(p) < 166.423$. The same error band gives

$165.422 < N(p) - p < 166.694$.

The only possible integer is 166. Therefore $N(10^{100}) = 10^{100} + 166$ is determined exactly.

With $n = 10^{100} + 166$, this enormous row again has the form

[1, n, n(n − 1)/2, …, n(n − 1)/2, n, 1].

Its exact digit count is $10^{100} + 167$. It represents $2^{10^{200} + 166 \times 10^{100}}$. The 1 at each end is an endpoint of the Pascal row. The … stands for specific omitted binomial coefficients of row n, not unknown or arbitrary values.

The use of N(p) is especially clear here: mathematics determines the carry-free boundary for a family of rows that cannot be printed in full. This does not claim that we stored the complete row or computed and displayed every entry.

Example 10. Boundary pairs for checking a program

p	N(p)	First row that is not carry-free
2	2	3
3	4	5
4	5	6
7	9	10
8	10	11
37	39	40
38	41	42
39	42	43
100	103	104
158	161	162
159	163	164
1000	1005	1006

Each table entry supplies two checks: row N(p) must fit, while row N(p) + 1 must not. These pairs can be used to check boundary handling in a base converter or a program that uses Pascal coefficients as digits. Changing p also changes the base. Jumps in this table do not mean that a row is skipped within one fixed base.

Scope of the uses

N(p) is a mathematical tool defining the exact valid range of this representation. It can illustrate carrying in teaching, guide base selection in an algorithm, and supply boundary checks. By itself, it does not show that generating a complete Pascal row or a very large power is faster. It does not provide a separate encryption method, a general compression result, or a primality test. The cost of computing N(p) also depends on the method used.

9. From addresses to cells: a PCCPU execution model

9.1. Architectural aim

The aim is to address numbers in a matrix or multidimensional arrangement of cells and perform arithmetic through those addresses. A cell need not hold only a number. It may represent a formula, a pending operation, a result, or the accessible state of a core. Attaching a local processing unit to a cell, or letting it represent a subordinate group of cells, are also design options.

In this view, core inputs, results and execution states become visible within a shared address scheme. PCCPU starts work, selects the required addresses and observes completion events. This connects the number-addressing study to the processor idea. The N(p) study is not the complete architecture. It supplies the carry-free boundary for the mathematical representation.

The existing PCCPU document describes a controller that distributes work and allows cores to operate independently [4]. The shared cell address and small execution example here are a proposed model for explaining that idea. They are not presented as multicore hardware implemented in Hunter or as a completed instruction set.

9.2. What can be found at an address?

The idea that a cell can contain a processor has two levels. In a physical design, memory and a processing unit could be placed together in a cell. In a logical model, the cell address identifies a core’s work record and state. The example below implements the second level. It does not determine a physical circuit layout.

Cell field	Purpose
Kind	Data, worker, controller or unused cell
Mathematical address	For example C(9,3), coefficient 3 of row 9
Operation and inputs	Produce a coefficient, add two ready values, write a destination
Owner	Logical core performing the work
State	Waiting, completed, or result ready
Value	Produced number; empty until it is produced

A RAM mapping provides access to these records by address. Addressing a record does not by itself make ordinary RAM perform computation. A processing unit and a result-write rule are also required. Still, making data and execution state visible through one address scheme is a concrete part of the design.

9.3. A small two-row cell arrangement

The example has two rows and three columns. Cell record addresses are chosen as 1000 + 3r + c. Here r and c are matrix coordinates, not the Pascal row n and coefficient position k. Addresses identify records. A byte width and physical memory layout are not specified.

Worker A at address 1000 produces C(9,3) = 84. Worker B at 1001 produces C(9,4) = 126. Worker C at 1002 takes these two records as inputs. Once both are ready, it adds them and writes 210 to address 1003. PCCPU state is visible at 1004. The last cell is unused.

Here, arithmetic between processors means connecting values produced by separate workers through addresses. It does not mean adding core identifiers. A and B may complete in different orders. C must wait for both inputs to be ready.

9.4. Counter, address and completion

The controller record in this small model holds three distinct items: a logical evaluation round, a completed-work event count, and the address of the last selected work cell. These are not final PCCPU register names or bit layouts. They illustrate one way to track order, address and state together.

Stage	Work in the cells	Visible controller state
Start	A, B and C wait. The result is empty.	Event count 0, no selected address
First ready set	A produces 84. B produces 126. C does not run yet.	Two completion events are recorded
Second ready set	C computes 84 + 126 = 210 and writes the destination.	The third event is recorded
Finish	The result at 1003 is ready.	The job is complete and is not started again

Two logical rounds do not mean two physical clock cycles. The example runs sequentially in Python. Selecting ready work from the same state snapshot makes the dependency visible. Hardware computation latency, communication, memory access and synchronization require separate design.

PCCPU need not centrally control every arithmetic step of every core. As in the existing design, it can distribute work and let cores advance with local counters [4]. The counter and address record in this model illustrate that aim. They do not prove that the controller will avoid becoming a bottleneck.

9.5. When can arithmetic become an address transformation?

The example gives more than the numeric result 210. There is also a Pascal-address relationship:

$C(9,3) + C(9,4) = C(10,4) = 210$.

The general rule is $C(n,k) + C(n,k+1) = C(n+1,k+1)$. Thus the sum of two suitable adjacent coefficients can be expressed by the destination coefficient's address. A system could store this relationship instead of immediately computing the value. The small execution model actually generates and adds the numbers so the example remains directly checkable.

Not every sum of two addresses becomes a single Pascal address. When no suitable relationship applies, a sum expression or an ordinary numerical result must be retained. Adding physical RAM addresses is not an implementation of the mathematical rule either. The objects and operations represented by the addresses must be known.

There is another rule for whole-row numbers in the same base. Define $R(b,n) = (b + 1)^n$. Then $R(b,r) \times R(b,s) = R(b,r+s)$. In base ten, multiplying the numbers for rows 2 and 2 gives the number for row 4: $121 \times 121 = 14641$. Rows 3 and 2 give $1331 \times 121 = 161051$.

The second product is numerically correct, but the coefficients of Pascal row 5, [1, 5, 10, 10, 5, 1], are not decimal digits. Carrying is required. In a Mersenne base $b = 2^p - 1$, reading the coefficients directly as digits requires $r + s \le N(p)$. Base ten is not a Mersenne base; its check is the general central-coefficient condition $C(r+s,\lfloor (r+s)/2 \rfloor) < 10$.

9.6. The role of N(x,y,z)[a]

N(x,y,z)[a] is a proposed notation for addressing row, input and position relationships with a direction selector. The intended choices for [a] are 0 for selection from the left and 1 from the right. $z = 0$ may select the center. In the examples discussed, x counts digits, so the Pascal row index is $x - 1$. The input value 1 in the y field is not identified with the base. The complete field meanings and data types are not yet fixed.

Reading a digit, raising it to its position, and computing its positional contribution are separate operations. Reading either 4 in 14641 returns 4. If its zero-based position from the left is used as the exponent, the results are 4^1 and 4^3. Their decimal positional contributions are instead 4×10^3 for the left 4 and 4×10^1 for the right 4. Changing the direction bit alone does not specify which operation is intended. The operation type must be explicit.

The small software model therefore does not implement the unsettled N notation as an instruction standard. It uses C(n,k) for the mathematical address, a matrix coordinate for an execution cell, and an address number for record access. Encoding these in a shared address word is a later design decision.

9.7. Running the concrete example

From the repository root, run:

```
python3 examples/pccpu_address_model.py
```

The example shows values produced by workers A, B and C, the destination address and controller state. The result is 210. A result cannot be read before it is ready, and completed work is not executed twice. The row-product examples separately report whether the resulting representation is carry-free.

This is an explanatory software model. It does not include a physical inter-core bus, cache coherence, interrupt mechanism, memory protection or clock circuit. A physical implementation needs defined cell widths, a readiness protocol, rules for multiple writers and conditions for advancing counters.

The mathematical basis is the set of valid transformations between addresses. The architectural proposal combines value and work state in a cell arrangement. N(p) has the concrete role of checking the carry-free boundary of the chosen representation. Together these can form an execution model; speed and energy advantages require separate evidence.

10. Reproduction and conclusion

The main Python and C programs generate row n = p + 2. This explanation does not change their behavior. N(p) checks use a separate verifier. From the repository root:

```
python3 mersenne_rows.py 7 full
python3 -B tests/verify_np.py --quick
python3 -B -m unittest discover -s tests -p 'test_*.py'
```

The full mathematical check is `python3 -B tests/verify_np.py`. Its direct scan covers $p \le 20000$ and its threshold checks cover $k \le 12$. A finite computational check is not a substitute for proof over all possible p.

The first study constructs selected carry-free number representations from Pascal rows. The N(p) extension determines the last valid row for this representation in a fixed base. The second study examines the boundary of the first. The central distinction is between finding a valid example and finding the last valid example.

References and terms of use

[1] James Augustus. *Pascal rows in Mersenne bases*. Original note: English text and English PDF. Repository: pascal-mersenne-rows.

[2] James Augustus. *The largest carry-free row in a Mersenne base*. N(p) extension: English text and English PDF.

[3] *N(p) computational checks*. Verifier. Direct scan $p \le 20000$, threshold checks $k \le 12$.

[4] James Augustus. PCCPU design note. Work distribution, core-local counters and open architectural decisions.

This explanation is covered by the repository's documentation license, CC BY 4.0. Copyright © 2026 James Augustus. The thesis-style layout describes the presentation only. This document is not described as a graduate thesis submitted to or approved by a university.